

Fraud Detection with Multi-Agent Learning Loop

A MongoDB Atlas Reference Architecture for
Real-Time AI-Powered Fraud Investigation

Powered by MongoDB Atlas

September 2026

Executive Summary

Financial institutions lose billions annually to fraud, yet most fraud detection systems suffer from two fundamental gaps: (1) they flag transactions but leave the investigation entirely to humans, and (2) they never learn from the outcomes of those investigations. This paper presents a reference architecture built entirely on MongoDB Atlas that closes both gaps — combining real-time stream processing, multi-agent AI investigation, and a closed-loop learning cycle that improves with every human review.

The system demonstrates six MongoDB Atlas capabilities working as a single platform: Atlas Stream Processing for real-time scoring, Atlas Vector Search with autoEmbed for semantic case retrieval, Atlas Search for lexical keyword matching, Hybrid Search via `$rankFusion` for best-of-both retrieval, the document model for rich nested investigation records, and the aggregation framework as the execution engine for natural-language-to-MQL query generation.

The result: a fraud operations system that investigates cases autonomously, routes decisions intelligently, learns from human feedback, and gets smarter over time — all on a single database platform with no additional infrastructure.

The Business Problem

Challenge	Current State	What's Needed
Volume	Thousands of flagged transactions per day, limited analyst capacity	Automated investigation that pre-analyzes cases before humans see them
Consistency	Each analyst investigates differently — no shared learning	A system that accumulates institutional knowledge from every review
Speed	Manual investigation takes 15-45 minutes per case	Pre-investigated cases with precedent briefs delivered in seconds
Learning	Lessons from past cases stay in analysts' heads	A memory system that retrieves relevant past conclusions for each new case
Trust	AI decisions are black boxes — humans can't verify or correct	Human-in-the-loop feedback that creates ground-truth signal for future decisions

The Solution: Three Surfaces, One Platform

1. Transaction Simulator

Injects test transactions with preset fraud scenarios (all indicators, unusual location only, high amount only, new device only, clean baseline). Each submission writes to the `transactions` collection and triggers the stream processing pipeline for real-time scoring.

2. Fraud Investigator (Agent)

Displays flagged cases and launches a multi-agent investigation graph that runs autonomously: retrieves precedents from past cases, investigates with tool-calling AI, makes a routing decision, and presents findings with a confidence level. A human analyst reviews the decision and provides feedback — triggering the learning cycle.

3. Interactive Analyst (Chatbot)

A conversational interface where analysts ask natural-language questions. The agent generates and executes MongoDB aggregation pipelines directly against the database — translating "show me fraud trends by country this week" into a live MQL pipeline. Supports hybrid search (vector + lexical via `$rankFusion`), vector-only, and lexical-only modes.

MongoDB Atlas Capabilities Demonstrated

1. Atlas Stream Processing — Real-Time Fraud Scoring

A continuous aggregation pipeline monitors the `transactions` collection's change stream. Every inserted transaction is scored in real time — no application code in the hot path.

- `$source` connects to `fraud_detection.transactions` on insert/update/replace
- `$lookup` joins `user_profiles` to get behavioral baselines
- Three fraud signals evaluated using `$cond` + `$concatArrays`:
 - `high_amount`: amount > 3x user's average (25 points)
 - `unusual_location`: country not in user's frequent locations (30 points)
 - `new_device`: device ID not in user's known devices (20 points)
- `$merge` writes results back to `transactions` with status, score, and indicators
- A second pipeline copies flagged transactions to `fraud_transactions`

MongoDB Value

No separate streaming platform (Kafka, Flink, Spark Streaming). The same database that stores the data also processes the stream — one platform, one connection, one operational model.

2. Atlas Vector Search with autoEmbed — Semantic Case Retrieval

The Investigation Agent and Analyst Chatbot search for similar past fraud cases using natural language and get semantically matching results without any client-side embedding logic.

- An autoEmbed vector index on `fraud_transactions.text` uses the `voyage-4` model
- Atlas generates and stores embeddings at write time — the application never calls an embedding API
- At query time, `$vectorSearch` accepts plain text
- No Voyage API key in the application, no embedding pipeline, no vector storage management

MongoDB Value

Traditional vector search requires a separate vector database (Pinecone, Weaviate, Milvus), an embedding service, and sync logic. With autoEmbed, the operational database *is* the vector database — embeddings are generated and stored automatically, and vector search runs alongside regular queries on the same documents.

3. Atlas Search (BM25) — Lexical Keyword Search

Enables precise keyword matching over merchant names, locations, device types, and fraud indicators — "show me all cases involving Electronics Store in Lagos."

- A static-mapping Atlas Search index with BM25 scoring on key fields
- `$search` operator with `text` query over multiple paths
- Returns relevance-scored results for exact term matching

MongoDB Value

Lexical search lives next to vector search on the same collection — no separate Elasticsearch cluster. The same query can combine `$search` (lexical) with `$vectorSearch` (semantic) in a single pipeline.

4. Hybrid Search via `$rankFusion` — Best-of-Both Retrieval

Combines semantic vector search and lexical BM25 search in a single query, fusing results via Reciprocal Rank Fusion (RRF) with configurable weights (60% vector, 40% lexical).

```
{
  $rankFusion: {
    input: {
      pipelines: {
        vectorPipeline: [{ $vectorSearch: { ... } }],
        lexicalPipeline: [{ $search: { ... } }, { $limit: 20 }],
      },
    },
    combination: { weights: { vectorPipeline: 0.6, lexicalPipeline: 0.4 } },
  },
}
```

MongoDB Value

No other database natively combines vector and lexical search with rank fusion in a single query. Competing solutions require application-level merging of results from two separate systems. With `$rankFusion`, Atlas handles the fusion server-side — one query, one result set, one round trip.

5. Document Model — Rich Investigation Records

Each fraud transaction is a single rich document containing the transaction details, fraud score, indicators, device metadata, merchant info, and a nested `investigation` subdocument that grows as the agent investigates and the human reviews.

```
{
  "transaction_id": "txn_demo_1234",
  "amount": 4800,
  "fraud_score": 75,
  "fraud_indicators": ["high_amount", "unusual_location", "new_device"],
  "merchant": { "merchant_name": "Unknown Electronics", "merchant_country": "NG" },
  "location": { "country": "NG", "city": "Lagos" },
  "device": { "device_id": "dev_unknown_001", "device_type": "mobile" },
  "investigation": {
    "confidence": "high",
    "recommendation": "escalate",
    "routed_to_human": true,
    "decision_rationale": "Pattern matches confirmed account takeover...",
    "notes": ["Device not in known devices", "Amount exceeds account limit"],
    "human_outcome": "confirmed_fraud",
    "human_reviewed_at": "2026-09-22T21:37:19Z",
    "precedent_brief": "PRECEDENT BRIEF for txn_demo_1234..."
  }
}
```

MongoDB Value

No joins. No object-relational mapping. The entire investigation — from scoring to agent findings to human feedback — is one document that evolves over time. A relational schema would require 5-7 tables with foreign keys to represent the same data.

6. Aggregation Framework as AI Execution Engine

The Analyst Chatbot's `run_aggregation` tool lets the AI model generate MongoDB aggregation pipelines from natural language and execute them directly against the database.

- Analyst asks: "What's the average fraud score by country this month?"
- Model generates: `[{ $match: { timestamp: { $gte: ... } } }, { $group: { _id: "$location.country", avgScore: { $avg: "$fraud_score" } } }]`
- Atlas executes it and returns the result set to the model
- Model summarizes the findings in plain English

Safety: blocks `$out`, `$merge`, `$indexStats`. Results capped at 50. Only allowed collections are queryable.

MongoDB Value

The aggregation framework is the most expressive query language in the database industry. By exposing it to an AI agent, the system turns natural language into live analytics — without pre-built dashboards, without BI tools, without a separate analytics database.

The Multi-Agent Investigation Graph

The investigation is orchestrated as a LangGraph state graph with three sequential nodes. Unlike prompt-enforced protocols (where the AI might skip steps), the graph topology guarantees execution order.

Node 1: Memory Retrieval (deterministic, no LLM)

1. Fetches the flagged transaction from `fraud_transactions`
2. Constructs a pattern description from indicators, amount, location, device, merchant
3. Runs **two parallel `$vectorSearch` calls** on `agent_memory` :
 - **Verified search:** filtered to `source: "human_confirmed"`
 - **Unverified search:** all memories
4. Computes an **accuracy stat**: aggregates `lesson_learned` docs with matching indicators
5. Assembles a structured **precedent brief**

Node 2: Investigation Agent (LLM – Claude Sonnet)

Receives the precedent brief as input and investigates with four tools:

Tool	MongoDB operation
<code>fetch_fraud_transaction</code>	<code>findOne</code> on <code>fraud_transactions</code>
<code>fetch_user_history</code>	<code>findOne</code> + <code>find</code> on <code>user_profiles</code> and <code>transactions</code>
<code>search_similar_fraud_cases</code>	<code>\$vectorSearch</code> on <code>fraud_transactions</code>
<code>submit_investigation_findings</code>	Returns to graph state

Node 3: Decision Agent (LLM – Claude Sonnet)

Applies a decision policy using the accuracy stat:

Condition	Decision
Accuracy < 60%	Force human escalation
Accuracy >= 75% + confidence "high" + auto_close	Auto-close
Confidence "medium" or "low"	Always escalate
No accuracy stat (first case)	Follow recommendation, lean toward escalation

The Learning Loop

When a human analyst reviews the agent's decision and clicks "Confirm fraud" or "False positive," a learning graph runs asynchronously:

- 1. **Ingest** (no LLM): Reads the case, compares agent vs human conclusion, produces `agreement` boolean
- 2. **Synthesize** (LLM): Claude generates a 2-4 sentence reusable lesson. Agreement → "confirmed pattern." Disagreement → "correction."
- 3. **Write Lesson** (no LLM): Inserts `lesson_learned` to `agent_memory` with `source: "human_confirmed"` . autoEmbed indexes it automatically.

The next investigation with similar indicators retrieves this lesson via `$vectorSearch` , includes it in the precedent brief, and the investigation agent reads it before deciding. The accuracy stat compounds with each review.

The Trust Gradient

The `source` field creates a trust gradient: `"agent"` = unverified prior reasoning (precedent, not fact). `"human_confirmed"` = ground truth. The Memory Retrieval Agent runs two parallel searches — one filtered to human-confirmed only — and the system prompt coaches the agent to prefer human-confirmed results.

How Accuracy Compounds

After N feedbacks	Accuracy	Decision Agent behavior
1 feedback, 0 agreed	0%	Force escalation (< 60%)
2 feedbacks, 1 agreed	50%	Force escalation (< 60%)
3 feedbacks, 2 agreed	67%	Follow recommendation (60-75%)
5 feedbacks, 4 agreed	80%	Allow auto-close (>= 75%)
10 feedbacks, 9 agreed	90%	Allow auto-close (>= 75%)

The system adjusts its own autonomy threshold based on past performance. Patterns where the agent is frequently wrong get forced to human review. Patterns where it's consistently right can be auto-closed.

MongoDB Value Framework

Speed to Value

Traditional approach	This project
Separate streaming platform (Kafka + Flink)	Atlas Stream Processing — same database
Separate vector database (Pinecone/Weaviate)	autoEmbed — Atlas generates embeddings automatically
Separate search engine (Elasticsearch)	Atlas Search — BM25 on the same collection
Application-level result merging for hybrid search	<code>\$rankFusion</code> — server-side fusion in one query
ETL pipeline to sync data to analytics warehouse	Aggregation framework — analytics on live data
5-7 infrastructure components	1 platform: MongoDB Atlas

Simplification

Without MongoDB	With MongoDB Atlas
Operational DB + vector DB + search engine + streaming + embedding service	One Atlas cluster
5+ vendor relationships, billing models, support contracts	One vendor, one bill, one support team
Data sync pipelines between systems	No sync — all capabilities on the same data
Embedding service + API key management + pipeline	autoEmbed — server-side, zero config

Trust and Security

- **Human-in-the-loop:** Every AI decision can be reviewed, confirmed, or corrected
- **Trust gradient:** Memory distinguishes agent conclusions from human-verified outcomes
- **Sandboxed AI queries:** `run_aggregation` blocks write operations, caps results
- **Audit trail:** Every investigation, feedback, and lesson persisted with timestamps
- **Encryption:** Data encrypted at rest and in transit (Atlas built-in)

Cost Efficiency

Cost category	Traditional	This project
Infrastructure	5+ separate services	1 Atlas cluster
Embedding API calls	Per-insert + per-query	Zero (autoEmbed is server-side)
Streaming platform	Kafka cluster + Flink compute	Included in Atlas

Search engine	Elasticsearch cluster	Included in Atlas
Vector database	Pinecone/Weaviate subscription	Included in Atlas

Key Takeaways

1. **One platform, six capabilities:** Stream Processing, Vector Search (autoEmbed), Atlas Search (BM25), Hybrid Search (`$rankFusion`), document model, aggregation framework — all on one cluster.
2. **The database is the AI infrastructure:** Vector embeddings, semantic search, lexical search, and real-time scoring all happen inside MongoDB. No external AI infrastructure beyond the LLM API.
3. **The system learns from humans:** Every human review produces a searchable lesson that improves future investigations. The accuracy stat controls how much autonomy the agent gets — it knows its own limits.
4. **Natural language to MQL:** Analysts ask questions in English. The AI generates MongoDB aggregation pipelines. Atlas executes them. No BI tool required.
5. **Simplification at every layer:** One vendor, one connection string, one query language, one deployment model — replacing 5+ infrastructure components with a single Atlas cluster.

Technical Stack

Component	Technology
Database + Search + Streaming	MongoDB Atlas
Vector embeddings	autoEmbed with voyage-4 (server-side)
AI model	Claude Sonnet 4 (Anthropic)
Agent orchestration	LangGraph (StateGraph)
Agent tools	LangChain tool() with Zod schemas
Web server	Express.js
Frontend	Vanilla HTML/CSS/JS with Server-Sent Events
Driver	mongodb Node.js driver v6

This paper describes a reference architecture for demonstration purposes. Production deployments should add rate limiting, authentication hardening, connection pooling, and monitoring.

Powered by MongoDB Atlas.